

Dream-SLAM: Dreaming the Unseen for Active SLAM in Dynamic Environments

Abstract—In addition to the core tasks of simultaneous localization and mapping (SLAM), active SLAM additionally involves generating robot actions that enable effective and efficient exploration of unknown environments. However, existing active SLAM pipelines are limited by three main factors. First, they inherit the restrictions of the underlying SLAM modules that they may be using. Second, their motion planning strategies are typically shortsighted and lack long-term vision. Third, most approaches struggle to handle dynamic scenes. To address these limitations, we propose a novel monocular active SLAM method, Dream-SLAM, which is based on *dreaming* cross-spatio-temporal images and semantically plausible structures of partially observed dynamic environments. The generated cross-spatio-temporal images are fused with real observations to mitigate noise and data incompleteness, leading to more accurate camera pose estimation and a more coherent 3D scene representation. Furthermore, we integrate dreamed and observed scene structures to enable long-horizon planning, producing farsighted trajectories that promote efficient and thorough exploration. Extensive experiments on both public and self-collected datasets demonstrate that Dream-SLAM outperforms state-of-the-art methods in localization accuracy, mapping quality, and exploration efficiency. Source code is available on our project website <https://anonymous.4open.science/r/Dream-SLAM>.

Index Terms—Active SLAM, dreaming, dynamic environments, Gaussian splatting.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) addresses the joint estimation of the ego-pose of a moving platform and a representation of its surrounding environment, and constitutes a fundamental building block for a wide range of emerging applications, including virtual, augmented, and mixed reality, as well as autonomous robots. In its most general formulation, the SLAM estimates are passively updated using measurements acquired from the platform’s onboard sensors [1], which may limit the accuracy, completeness, and task-specific relevance of the resulting map. In contrast, robotic platforms can actively control their motion to deliberately gather informative measurements that improve the quality of the estimated representation. This paradigm, commonly referred to as active SLAM [2], has enabled significant advances in applications such as search-and-rescue operations, warehouse inventory management, and large-scale scene reconstruction [3].

Despite remarkable progress in recent years, active SLAM remains constrained by three major limitations. First, its overall performance is strongly conditioned by the quality of the underlying estimation modules. Most research, however, focuses primarily on the action or planning components, while relying on off-the-shelf localization and mapping strategies [4], [5], or evaluating exploration performance using ground-truth camera poses [6], [7]. Since localization and mapping constitute the backbone of active SLAM,

improvements in these components directly translate into gains across the entire pipeline. Second, from a planning perspective, the majority of methods adopt either frontier-based [8] or sampling-based [9] strategies. While effective in certain scenarios, these planners are inherently shortsighted: they operate solely on the currently observed map and lack mechanisms to reason about or anticipate unexplored regions. As a consequence, they often converge to locally optimal trajectories with unnecessary detours or frequent backtracking. Third, most methods [10], [11] assume a static environment, which rarely holds in practice. In dynamic scenes, such as homes with moving occupants or crowded shopping malls, the presence of dynamic objects introduces occlusions and induces localization drift, both of which significantly degrade planning reliability and overall performance.

In recent years, there have been efforts to mitigate the aforementioned limitations. First, for more accurate localization and mapping, some methods [12], [13] encourage robots to preferentially move to areas with rich visual textures. This strategy is motivated by the observation that textured areas often provide distinctive and reliable features useful for camera pose estimation and 3D reconstruction. However, prioritizing highly textured areas may introduce exploration bias, leading to incomplete environment coverage as low-texture areas may be systematically neglected. Second, the shortsighted planning can be alleviated by reasoning over unknown space to generate global coverage paths [14], [15]. However, these approaches often assume that unexplored regions exhibit regular layouts, resulting in an overly simplified approximation. Third, to tackle dynamic scenes, several methods [16], [17] explicitly filter out moving objects to maintain a static background map, discarding informative foreground content. More recent work [18] can reconstruct the dynamic foreground, but tends to become unstable under fast or complex motions, and also significantly increases system complexity.

To address the above challenges, we propose a monocular active SLAM method, named **Dream-SLAM**, via dreaming the unseen in dynamic environments. The core idea of our method is dreaming the cross-spatio-temporal images and the semantically plausible structures of the scene. For one thing, the cross-spatio-temporal images can be combined with real images to compensate for noise and data incompleteness. This combination leads to a more accurate camera pose estimation, as well as a more coherent 3D scene representation. For another, we integrate the dreamed and observed scene structures for long-horizon planning. This way generates a farsighted path that can efficiently achieve a thorough exploration. As shown in Fig. 1, our method is composed of two main modules for localization and mapping, as well as exploration planning. The localization and mapping module supplies the observed

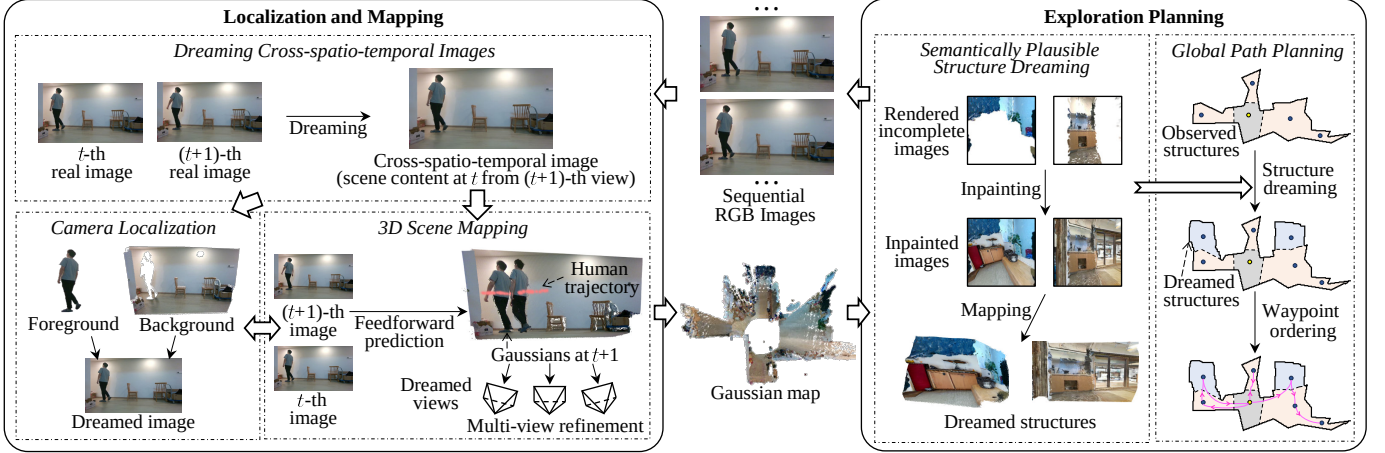


Fig. 1. **Dream-SLAM overview.** Our pipeline consists of two main modules: localization and mapping, and exploration planning. (a) For localization, we propose to dream cross-spatio-temporal images, and use these images to construct additional 3D-2D foreground constraints that can effectively compensate for noise. For mapping, we propose a feedforward network to reconstruct per-pixel Gaussians of both static background and dynamic foreground. We further refine Gaussians based on multi-view constraints provided by cross-spatio-temporal and real images. (b) Our planning module dreams semantically plausible structures of unobserved areas. By integrating the dreamed and observed information, we plan a farsighted path, enabling an efficient and thorough exploration.

environment information to the planning module. In turn, the planner provides motion commands that drive the robot to acquire new observations.

As to camera localization in dynamic environments, a common practice is to use 3D-2D constraints of the static background only. While the dynamic foreground has the potential to improve the accuracy, it can hardly be used in practice. The reason is that 2D image of the foreground at the current time is inconsistent with 3D foreground reconstructed at the previous time, due to object movement. To solve this problem, we propose to dream a cross-spatio-temporal image, which depicts the dynamic 3D scene at the previous time, from the viewpoint of the current camera. Based on the consistent 3D-2D foreground content at the previous time, we can establish constraints on the pose of the current camera. We dream the image by designing a network based on diffusion [19]. The dreamed image exhibits high appearance fidelity and spatial rationality. In addition, to enable photo-realistic and efficient mapping of dynamic scenes, we propose a feedforward network compatible with Gaussian splatting [20]. This network can directly predict per-pixel Gaussians of both dynamic foreground and static background. It is more concise and general than existing Gaussian estimation solutions [21], [22]. To further refine Gaussians, we leverage both dreamed cross-spatio-temporal and real images. The dreamed views can effectively supplement the real observations by providing additional supervisory signals.

In terms of planning, we aim to overcome the shortsightedness of conventional planners [6], [23]. First, on the waypoints that the robot will potentially visit, we place multiple virtual cameras. We use these cameras to render the reconstructed 3D scenes, obtaining a set of incomplete images. Then we introduce a diffusion-based model to inpaint these images, generating virtual observations of the unexplored region. We further back-project the inpainted images into 3D Gaussians based on the above feedforward network. By integrating these Gaussians with the existing map, we obtain a more complete

scene structure. The above dreaming process considers the observed information to infer the unseen content. Accordingly, the dreamed structures implicitly encode the information of the surroundings, and thus are semantically plausible. Planning within such structures leads to a farsighted path. Please note that the dreamed structures are not used for final mapping, but are replaced by the real observations once visited.

Our main contributions are summarized as follows:

- We introduce dreaming as a unified mechanism for localization, mapping, and exploration planning. The dreamed content effectively supplements the real observations, enhancing the performance of active SLAM.
- For localization, we propose to dream cross-spatio-temporal images. These images can incorporate the information of the dynamic foreground to compensate for noise and data incompleteness, improving the accuracy of camera pose estimation.
- For mapping, we propose a feedforward network to efficiently predict Gaussians, achieving a photo-realistic scene reconstruction. Then we refine Gaussians by the cross-spatio-temporal images, which can supplement real views and achieve a more coherent 3D representation.
- For planning, we propose to dream the semantically plausible structures of unexplored regions. By integrating the dreamed and observed structures, our method generates a farsighted path that can efficiently achieve a thorough and efficient exploration.

Extensive experiments on both public and self-collected datasets demonstrate the superior performance of our Dream-SLAM in localization and mapping accuracy, as well as exploration efficiency, compared to state-of-the-art approaches.

II. RELATED WORK

A. Passive SLAM

Geometric Representations. Dominant methods estimate camera poses by establishing point correspondences across

views and reconstruct sparse 3D point clouds through triangulation [24], [25]. Some methods extend the sparse feature points to all the image pixels, optimizing intensity and depth residuals to achieve dense scene representation [26], [27]. In recent years, several methods have incorporated deep neural networks to further enhance performance. For example, DROID-SLAM [28] employs a recurrent network to enhance camera pose estimation through jointly learned depth and correspondence cues. The above methods operate well in static environments, but their performance degrades when dynamic objects appear. To cope with dynamic scenes, some approaches directly detect and filter out foregrounds. Among them, DynaSLAM [29] and ReFusion [30] utilize geometric residual, FlowFusion [31] relies on optical flow estimation, and DS-SLAM [32] leverages semantic segmentation. The common limitation of these approaches is that they fail to reconstruct the dynamic foregrounds. To address this limitation, several methods have been proposed. VDO-SLAM [16] models the motion of rigid items by incorporating geometric constraints. AirDOS [33] and Body-SLAM [34] further model non-rigid human motion and reconstruct articulated skeletons. MonST3R [35] leverages the learned multi-view geometry prior to jointly estimate camera poses and dense 3D structure across views. Despite these advances, they cannot reconstruct dynamic objects with photo-realistic details.

Neural Representations. Neural representation, which typically includes implicit neural radiance fields [36] and explicit Gaussian splatting [20], has recently shown strong performance in photo-realistic mapping [37]. Early works focused primarily on the use of photometric constraints in static environments [38]–[41]. Recently, some works have extended these techniques to handle dynamic scenes. Rodyn-SLAM [17] and DG-SLAM [22] leverage optical flow and semantic information to estimate the mask of dynamic objects and filter them out. WildGS-SLAM [21] employs a network to learn uncertainty online, but this increases the computational complexity. These methods also cannot reconstruct the dynamic foregrounds, thereby neglecting potentially useful information. In contrast, PG-SLAM [18] leverages shape priors to reconstruct both rigid and non-rigid objects, while jointly using foreground and background information to localize the camera. However, in highly dynamic scenes, foreground reconstruction can be unreliable, and separate reconstructions for rigid and non-rigid objects further increase system complexity.

Overall, the above classical SLAM methods neither effectively leverage the foreground information to localize the camera, nor reliably reconstruct the dynamic foreground. We address these limitations from two aspects. For one thing, we dream cross-spatio-temporal images, establishing foreground-related constraints. For another, we introduce a feedforward network that can directly predict per-pixel Gaussians of both dynamic foreground and static background.

B. Active SLAM

Sampling-based Exploration. The sampling-based methods generate multiple waypoints within the scene, which encode position and orientation information. Among these

candidate waypoints, the robot selects the optimal one. Bircher et al. [9] generate a rapidly-exploring random tree within the space, and then under a receding-horizon scheme, select the branch with the most unmapped space to explore. ANM [7] employs a neural scene representation and selects viewpoints according to the uncertainty distribution of this neural map. ActiveGS [42] reconstructs a Gaussian map, considering both rendering uncertainty and path cost. ActiveGAMER [43] incorporates a dynamically updated candidate pool to manage waypoints, reducing computational cost. The above methods partly neglect the spatial continuity between neighboring waypoints. Accordingly, the planning is prone to getting stuck into a local optimum, resulting in redundant detours and backtracking. To solve this problem, the recent ActiveSplat [6] introduces a topological graph to cluster the waypoints into multiple sub-regions. When making the next-step decision, it prioritizes the exploration completeness of each sub-region based on information gain and distance.

Frontier-based Exploration. The frontier-based exploration was first introduced in [8] and later systematically formalized in [44]. Frontiers are defined as the boundaries between occupied and unknown regions. In this paradigm, the robot selects the closest frontier as the next exploration target. Cieslewski et al. [45] select the frontier that is within the current field of view and impose minimal impact on the speed of flight, ensuring stable and efficient exploration at high velocities. Shen et al. [46] employ a stochastic differential equation-based strategy to select the region with the strongest particle expansion as the next frontier. FisherRF [23] represents the scene with 3D Gaussians and selects frontier-related waypoints by maximizing Fisher information. The above methods commonly adopt a greedy strategy, which tends to a locally maximum information gain. To address this, some methods incorporate global information into the planning process. For example, FUEL [47] adopts a hierarchical planning strategy to obtain a global exploration path. FALCON [48] decomposes the unknown space into multiple disjoint zones, and subsequently plans a path to effectively cover all of them. In addition to the observed information, recent methods [14], [15] alleviate the shortsighted planning by reasoning over unknown space. However, they often assume that unexplored regions exhibit regular layouts, resulting in an overly simplified approximation.

Overall, the above active SLAM approaches either rely solely on local observations, or infer an unrealistic approximation of unexplored areas. These limitations lead to a locally optimal path. By contrast, our method dreams the semantically plausible structures of unexplored regions. Accordingly, our planner can reason over a more complete scene layout, producing a more farsighted path.

III. PROBLEM FORMULATION

A. Preliminaries

Diffusion Models. Diffusion model [19] is a powerful image generation tool. The forward process gradually adds Gaussian noise to the latent code that encodes the input image, while the reverse process learns to remove this noise step

by step. At a certain step s , given a noisy latent code $\mathbf{z}_s$, a network $\mathcal{D}$ is used to predict the noise as $\epsilon = \mathcal{D}(\mathbf{z}_s, s)$. The predicted noise ϵ is a function with respect to the network $\mathcal{D}$. We can train $\mathcal{D}$ via the loss $\mathcal{L} = \mathbb{E}[\|\hat{\epsilon} - \epsilon(\mathcal{D})\|_2]$, where $\mathbb{E}$ represents the expectation, and $\hat{\epsilon}$ denotes the ground-truth added noise.

3D Foundation Models. We introduce a representative model DUS3R [49] that can produce a 3D point cloud from images in static environments. The network first encodes two consecutive images separately to obtain image features. These features are then fed to two branch decoders, each of which consists of multiple blocks. Based on the cross-attention, decoders generate image tokens. Finally, a point cloud head of the first branch takes corresponding tokens as input and predicts the point cloud in the first camera frame. The second branch follows the same process to obtain a point cloud in the first camera frame.

Gaussian Splatting. A 3D scene can be represented in a photo-realistic way through a set of 3D Gaussians $\mathcal{G}$ [20]. Each Gaussian is parameterized by its center, covariance matrix, opacity, and color. Through differentiable rendering $\pi[\cdot]$, an image can be obtained as $\tilde{I} = \pi[\mathcal{G}]$. To optimize Gaussians $\mathcal{G}$, we minimize the appearance difference between the rendered image $\tilde{I}$ and the ground-truth image I based on the photometric loss: $\mathcal{L}_{\text{photo}}(I, \tilde{I}) = \alpha \cdot \|I - \tilde{I}\|_1 + (1 - \alpha) \cdot (1 - \text{SSIM}(I, \tilde{I}))$, where α controls the trade-off between the L_1 and SSIM terms [50].

B. Dream-SLAM Overview

As shown in Fig. 1, our Dream-SLAM takes RGB images as input and does not rely on depth images. It consists of two modules for localization and mapping, as well as exploration planning. We segment dynamic objects in each image based on Mask R-CNN [51]. Our approach can handle both rigid objects (e.g., boxes) and non-rigid objects (e.g., humans and animals). Without loss of generality, we focus on humans as the main illustrative example, while experiments on other object categories are reported in the supplementary material. Moreover, we follow [18] to perform bundle adjustment and loop closure for optimization.

Localization and Mapping. For localization, given two images obtained at the previous and current times, we employ a diffusion model to dream a cross-spatio-temporal image. Such an image depicts the previous 3D scene from the current viewpoint. Using both the foreground and background of this image as supervision, we define photometric constraints to optimize the camera pose. To efficiently map both foreground and background in a photo-realistic manner, we design a feedforward network that can directly estimate pixel-wise Gaussians. In addition, we optimize Gaussians using multi-view photometric constraints provided by not only the real images, but also the dreamed cross-spatio-temporal images. Details are available in Section IV.

Exploration Planning. We construct a 2D topological map from the Gaussian map described above, representing both waypoints and their connectivity. The robot iteratively selects the optimal waypoint to move. Differently from existing short-sighted planners, we propose to dream semantically plausible

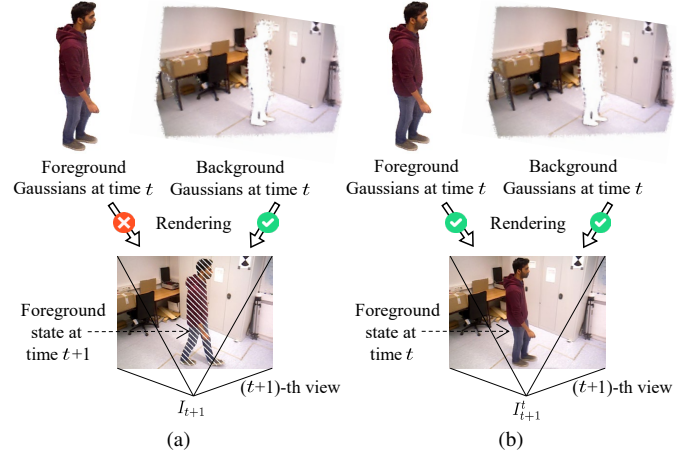


Fig. 2. **Cross-spatio-temporal images for camera localization.** (a) Traditional localization methods rely solely on the static background to estimate the camera pose. (b) In contrast, our method leverages both the dynamic foreground and static background by aligning the Gaussians' rendering at time t with the dreamed cross-spatio-temporal image I_{t+1}^t , which represents the scene at time t from the viewpoint of the $(t+1)$ -th camera.

structures of unexplored areas, followed by integrating the observed and dreamed information for long-horizon planning. Specifically, we first employ the diffusion model to inpaint the missing content of images rendered at unvisited waypoints. Then we use a set of inpainted images to reconstruct Gaussians of the unobserved areas, and integrate them into the existing Gaussian map. Based on the enriched map, we further update the topological map, in which we plan a farsighted path. Details are available in Section V.

IV. LOCALIZATION AND MAPPING

In this section, we introduce how we overcome the challenges raised by dynamic objects in localization and mapping. We propose to dream cross-spatio-temporal images that contribute to the accuracy improvement of localization, as well as a coherent 3D representation. Without loss of generality, let us consider the images I_t and I_{t+1} obtained at times t and $(t+1)$ for illustration.

A. Dreaming Cross-spatio-temporal Images

We begin by introducing the definition and role of cross-spatio-temporal images in the camera localization task. As illustrated in Fig. 2(a), Gaussian splatting-based localization methods typically estimate the pose of the image I_{t+1} by aligning it with a rendering of the Gaussians $\mathcal{G}_t$ reconstructed up to time t [40]. However, this strategy does not hold in the presence of dynamic foregrounds, as object states in the image I_{t+1} are inconsistent with those encoded in Gaussians $\mathcal{G}_t$.

To overcome this limitation, we propose to dream a cross-spatio-temporal image I_{t+1}^t (see Fig. 2(b)), which depicts the full scene (both dynamic foreground and static background) at time t , from the viewpoint of the $(t+1)$ -th camera. Accordingly, the object states in the image I_{t+1}^t and Gaussians $\mathcal{G}_t$ are consistent. We formulate the generation of I_{t+1}^t as an inpainting problem in two stages, as detailed below.

Generation of Inpainting Mask. As shown in Fig. 3, the real image I_{t+1} and the cross-spatio-temporal image I_{t+1}^t share the same background, as they are rendered from the same viewpoint, but differ in their foreground due to object motion. Accordingly, dreaming I_{t+1}^t amounts to replacing the foreground in the real image I_{t+1} with the foreground state at time t . Directly using the foreground segmented from the image I_t for this replacement is, however, not appropriate due to the change in viewpoint. Moreover, because of both object motion and viewpoint variation, parts of the foreground at time t may lie outside the foreground mask of I_{t+1} . To address these challenges, we define the inpainting mask $\mathbf{M}$ of the image I_{t+1} . This mask fully encloses both the foreground to be removed in I_{t+1} , as well as the foreground to be fused from time t . We obtain this mask $\mathbf{M}$ by performing a lightweight dilation to the foreground mask of I_{t+1} . Within this mask, foreground status in I_{t+1} is replaced by the foreground status at time t , while the background status in I_{t+1} remains unchanged. Experiments show that this mask design is robust across dynamic scenes with diverse object motions and scales.

Two-view-guided Image Inpainting. Given the images I_t and I_{t+1} , along with the inpainting mask $\mathbf{M}$, we adapt the diffusion model introduced in Section III-A to inpaint the masked regions of I_{t+1} .¹ We first describe the forward diffusion process. We assume that the ground-truth cross-spatio-temporal image $\hat{I}_{t+1}^t$ is available (details will be introduced below). We encode $\hat{I}_{t+1}^t$ together with the input images I_t and I_{t+1} , using a pretrained variational autoencoder [52], to obtain the latent codes $\hat{\mathbf{z}}_0$, $\mathbf{c}_1$, and $\mathbf{c}_2$, respectively. The codes $\mathbf{c}_1$ and $\mathbf{c}_2$ are then concatenated to form a reference code $\mathbf{c}$. Gaussian noise ϵ is progressively added to $\hat{\mathbf{z}}_0$, yielding the noisy latent $\mathbf{z}_s$ at step s . In the reverse diffusion process, we introduce a noise prediction network $\mathcal{D}$ to predict the added noise ϵ :

$$\epsilon = \mathcal{D}(\mathbf{z}_s, s, \mathbf{c}, \bar{\mathbf{M}}). \quad (1)$$

The network $\mathcal{D}$ is conditioned on two key inputs: the reference code $\mathbf{c}$, which encodes the unchanged context, and the inpainting mask $\bar{\mathbf{M}}$, which specifies the region to be inpainted. These conditioning signals enable context-aware inpainting, allowing the synthesized region to be seamlessly blended with its surroundings. To train the network $\mathcal{D}$, we use the following loss:

$$\mathcal{L} = \mathbb{E} \left[\|\bar{\mathbf{M}} \odot (\hat{\epsilon} - \epsilon(\mathcal{D}))\|_2 \right], \quad (2)$$

where $\odot$ denotes the element-wise multiplication. During inference, starting from a noisy latent initialized as white Gaussian noise, we can recover the noise-free latent code $\mathbf{z}_0$ through the reverse diffusion process. The recovered latent $\mathbf{z}_0$ is then decoded using the pretrained VAE decoder [52], yielding the inpainted image I_{t+1}^t .

In the following, we introduce the generation of the ground-truth cross-spatio-temporal image $\hat{I}_{t+1}^t$ for network training. To the best of our knowledge, no existing real-world SLAM dataset provides such images. While a simulator-based strategy is plausible, the domain gap between synthetic

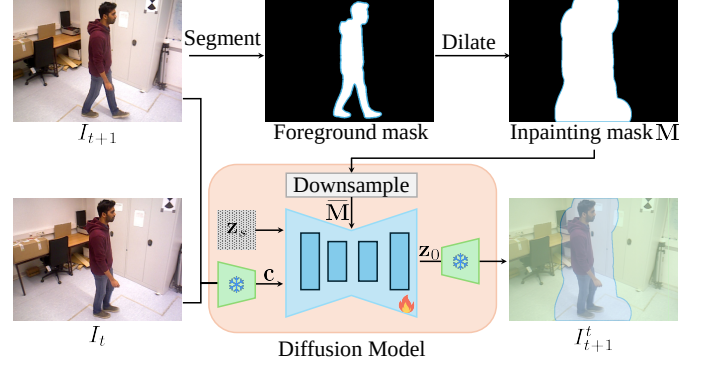


Fig. 3. **Dreaming a cross-spatio-temporal image.** Given the image I_{t+1} , we segment the foreground and dilate the foreground mask to obtain the inpainting mask $\mathbf{M}$. Then we feed the images I_t and I_{t+1} , together with the mask $\mathbf{M}$, into the diffusion model, which dreams the cross-spatio-temporal image I_{t+1}^t .

and real-world scenes makes the simulated images unsuitable for our task. To solve this problem, we adopt a 4D Gaussian splatting approach [53]. Given multiple real images $\{\dots, I_{t-1}, I_t, I_{t+1}, \dots\}$ obtained in a dynamic scene, we perform high-fidelity reconstruction of both static background and dynamic foreground. Then we render the reconstructed Gaussians at time t from the $(t+1)$ -th view into a photo-realistic image $\hat{I}_{t+1}^t$. We treat $\hat{I}_{t+1}^t$ as the ground-truth cross-spatio-temporal image.

B. Camera Localization

Our aim is to compute the pose $\mathbf{T}_{t+1}$ of the $(t+1)$ -th camera frame, given the estimate of the pose $\mathbf{T}_t$ of the t -th camera, and a scene representation at time t encoded as a set of Gaussians $\bar{\mathcal{G}}_t$.² We first estimate the relative transformation $\Delta\mathbf{T}_{t \rightarrow t+1}$ from the t -th to the $(t+1)$ -th camera frame leveraging photometric and geometric constraints, that we describe below. Once $\Delta\mathbf{T}_{t \rightarrow t+1}$ is obtained, $\mathbf{T}_{t+1}$ is obtained as $\mathbf{T}_{t+1} = \Delta\mathbf{T}_{t \rightarrow t+1} \mathbf{T}_t$.

Photometric Constraints. Recall that, typically, photometric constraints are only applied to the static scene parts (see Fig. 2(a)). In contrast, leveraging our dreamed cross-spatio-temporal images, we introduce a novel photometric constraint that incorporates the dynamic foreground, thereby providing richer supervisory signals. As shown in Fig. 2(b), we transform the foreground and background Gaussians $\bar{\mathcal{G}}_t$ at time t from the t -th camera frame to the $(t+1)$ -th one using the transformation $\Delta\mathbf{T}_{t \rightarrow t+1}$. Then we render these Gaussians from the $(t+1)$ -th view into an image:

$$\tilde{I}_{t+1}^t(\Delta\mathbf{T}_{t \rightarrow t+1}) = \pi[\bar{\mathcal{G}}_t, \Delta\mathbf{T}_{t \rightarrow t+1}]. \quad (3)$$

The rendered image $\tilde{I}_{t+1}^t(\Delta\mathbf{T}_{t \rightarrow t+1})$ depends on the transformation $\Delta\mathbf{T}_{t \rightarrow t+1}$. Since $\tilde{I}_{t+1}^t$ depicts both dynamic foreground and static background at time t from the $(t+1)$ -th view, it corresponds to the above cross-spatio-temporal image I_{t+1}^t .

¹To match the spatial resolution of the noise used in the diffusion process, the mask $\mathbf{M}$ is downsampled, yielding $\bar{\mathbf{M}}$.

²In our context, “-” (e.g., $\bar{\mathcal{G}}_t$ and $\bar{\mathcal{P}}_t$) represents prior information, which is used to distinguish it from the estimated value.

We use the images $\tilde{I}_{t+1}^t$ and I_{t+1}^t to formulate the photometric loss, optimizing the transformation $\Delta \mathbf{T}_{t \rightarrow t+1}$ as

$$\min_{\Delta \mathbf{T}_{t \rightarrow t+1}} \mathcal{L}_{\text{photo}}(I_{t+1}^t, \tilde{I}_{t+1}^t(\Delta \mathbf{T}_{t \rightarrow t+1})). \quad (4)$$

By jointly leveraging the information of both static and dynamic content, our approach significantly enhances localization accuracy, as validated through experiments. In practice, to conduct optimization in Eq. (4), we need a relatively reliable initial value of the transformation $\Delta \mathbf{T}_{t \rightarrow t+1}$. We obtain it based on the following geometric constraint.

Geometric Constraints. We feed the images I_{t+1} and I_t to our Gaussian prediction network introduced below. As partial output, we obtain a set of Gaussians $\mathcal{G}_t$, which corresponds to the 3D scene at time t , in the $(t+1)$ -th camera frame. Then we extract the centers of $\mathcal{G}_t$ and the above prior Gaussians $\bar{\mathcal{G}}_t$ respectively, obtaining point clouds $\mathcal{P}_t$ in the $(t+1)$ -th camera frame and $\bar{\mathcal{P}}_t$ in the t -th camera frame. Since $\mathcal{P}_t$ and $\bar{\mathcal{P}}_t$ are associated with the same image I_t , each pair of points is inherently associated through their corresponding pixel. Accordingly, we can establish a set of point correspondences $\{(\mathbf{p}_k, \bar{\mathbf{p}}_k)\}$ between $\mathcal{P}_t$ and $\bar{\mathcal{P}}_t$. Given these point correspondences, we formulate the following point cloud alignment loss based on the transformation $\Delta \mathbf{T}_{t \rightarrow t+1}$:

$$\min_{\Delta \mathbf{T}_{t \rightarrow t+1}} \sum_k \|\mathbf{p}_k - \Delta \mathbf{T}_{t \rightarrow t+1}(\bar{\mathbf{p}}_k)\|_2^2. \quad (5)$$

Specifically, $\Delta \mathbf{T}_{t \rightarrow t+1}$ is obtained via singular value decomposition (SVD) [54], and used as the initial seed for the photometric constraint-based optimization in Eq. (4). Compared with the existing methods targeting static environments [55], our strategy has two strengths. First, it leverages the dynamic foreground to better compensate for noise. Second, it leads to a higher degree of overlap between point clouds for the alignment, providing a higher number of constraints.

C. 3D Scene Mapping

We propose a novel network that can predict both foreground and background Gaussians in a feedforward manner. It can significantly improve the efficiency while maintaining high accuracy, compared with classical Gaussian splatting-based SLAM [18], [21]. Moreover, we leverage the dreamed cross-spatio-temporal images to further refine the Gaussians.

Feedforward Gaussian Prediction. Fig. 4(a) shows the architecture of our network, which consists of two modules. The first module regresses Gaussian positions, similarly to the foundation model introduced in Section III-A. In brief, given images I_{t+1} and I_t , we obtain two sets of image tokens $\{G_1^i\}_{i=1}^N$ and $\{G_2^i\}_{i=1}^N$. These tokens are fed to two position heads, outputting the Gaussian centers (point clouds), both in the $(t+1)$ -th camera frame. The second module, which is our main contribution, predicts the rest of the attributes for each Gaussian, using the Gaussian centers predicted by the first module as geometric guidance. Specifically, in this second module, we first leverage the Point Transformer V3 [56] in order to extract point cloud features. We then use N zero-convolution layers to map these features into point tokens $\{K_1^i\}_{i=1}^N$ and $\{K_2^i\}_{i=1}^N$. This mapping does not involve cross

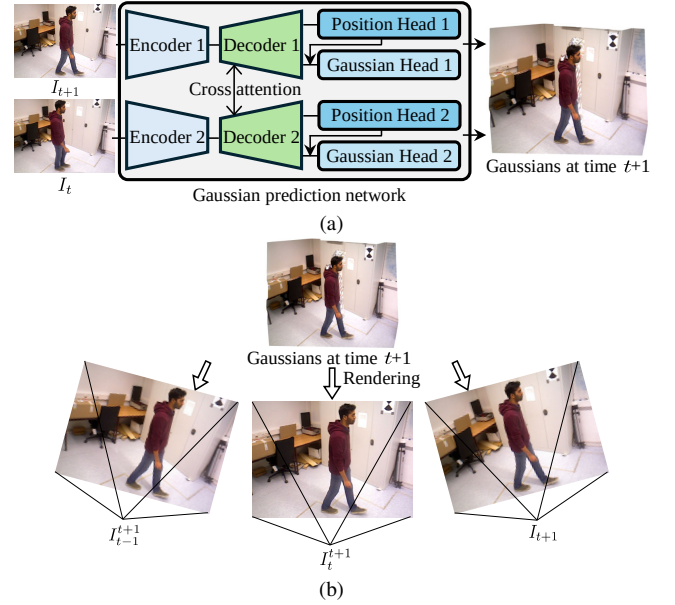


Fig. 4. **3D Gaussian prediction and refinement.** (a) Given images I_{t+1} and I_t , we design a feedforward network to predict dynamic Gaussians at both time $t+1$ and time t . Here, we only visualize the predicted Gaussians at time $t+1$. (b) We refine Gaussians at time $t+1$ based on the photometric loss regarding both dreamed cross-spatio-temporal images I_{t-1}^{t+1} , I_t^{t+1} and real image I_{t+1} . These images depict the same scene content at time $t+1$ from the $(t-1)$ -th, t -th, and $(t+1)$ -th views, respectively.

attention-based blocks, since the integration of this information has been completed before generating the point clouds. Further, taking the first branch as an example, we fuse the above point and image tokens by $\hat{G}_1^i = G_1^i + K_1^i$. Finally, we introduce Gaussian heads $\mathcal{H}_1$ and $\mathcal{H}_2$ to respectively predict Gaussian attributes $\mathcal{G}_{t+1}$ and $\mathcal{G}_t$, along with their associated confidences $\mathcal{C}_{t+1}$ and $\mathcal{C}_t$:

$$\mathcal{G}_{t+1}, \mathcal{C}_{t+1} = \mathcal{H}_1(\hat{G}_1^0, \dots, \hat{G}_1^N), \quad (6a)$$

$$\mathcal{G}_t, \mathcal{C}_t = \mathcal{H}_2(\hat{G}_2^0, \dots, \hat{G}_2^N). \quad (6b)$$

As the map grows, per-pixel Gaussian predictions may cause redundancy. In that case, we prune Gaussians based on their local density to minimize storage overhead. Notably, pruning is applied only to Gaussians that are no longer tracked, to avoid performance degradation.

We train our Gaussian prediction network $\mathcal{R}$ as follows. Firstly, we use the photometric loss to enforce the appearance constraint. Without loss of generality, we take the predicted Gaussians $\mathcal{G}_{t+1}(\mathcal{R})$ and their associated confidences $\mathcal{C}_{t+1}(\mathcal{R})$ in Eq. (6a) as an illustrative example. From the $(t+1)$ -th view, we render these Gaussians into an image $\tilde{I}_{t+1}$, which innovatively incorporates the confidences as weights:

$$\tilde{I}_{t+1}(\mathcal{R}) = \pi[\mathcal{G}_{t+1}(\mathcal{R}) \cdot \mathcal{C}_{t+1}(\mathcal{R})]. \quad (7)$$

We then optimize the weights of the network $\mathcal{R}$ by minimizing the difference between the rendered image $\tilde{I}_{t+1}(\mathcal{R})$ and the ground-truth image I_{t+1} to optimize the network $\mathcal{R}$, specifically $\min_{\mathcal{R}} \mathcal{L}_{\text{photo}}(I_{t+1}, \tilde{I}_{t+1}(\mathcal{R}))$. In addition, we extract point clouds from Gaussians, and apply the geometric loss [35] for network training.

Gaussian Refinement. The predicted Gaussians should be reliable. However, since the dynamic foreground at a given time is constrained by only a single view, there remains room for improving accuracy. For this reason, we refine the Gaussians based not only on the real images, but also the dreamed cross-spatio-temporal images. This refinement can use different numbers of dreamed images. We empirically observed that two dreamed images achieve the best balance between accuracy and efficiency, and thus we introduce this case in the following.

As shown in Fig. 4(b), our aim is to optimize the foreground and background Gaussians $\mathcal{G}_{t+1}$ at time $t + 1$ using multi-view constraints. Following Section IV-A, we first use the real image pairs (I_{t+1}, I_t) and (I_{t+1}, I_{t-1}) to dream the cross-spatio-temporal images I_t^{t+1} and I_{t-1}^{t+1} , respectively. These images depict the Gaussians $\mathcal{G}_{t+1}$ from the t -th and $(t - 1)$ -th views, respectively. Then, with the poses of the $(t + 1)$ -th and t -th cameras (obtained as described in Section IV-B), we transform the Gaussians $\mathcal{G}_{t+1}$ from the $(t + 1)$ -th camera frame to the t -th camera frame, and further render them into an image $\tilde{I}_t^{t+1}(\mathcal{G}_{t+1})$. Similarly, we also transform the Gaussians $\mathcal{G}_{t+1}$ to the $(t - 1)$ -th camera frame, and further render them into an image $\tilde{I}_{t-1}^{t+1}(\mathcal{G}_{t+1})$. These rendered images respectively correspond to the above cross-spatio-temporal images I_t^{t+1} and I_{t-1}^{t+1} , which can provide effective supervisory constraints on Gaussians $\mathcal{G}_{t+1}$. We formulate such constraints with the following photometric loss:

$$\min_{\mathcal{G}_{t+1}} \mathcal{L}_{\text{photo}}(I_t^{t+1}, \tilde{I}_t^{t+1}(\mathcal{G}_{t+1})) + \mathcal{L}_{\text{photo}}(I_{t-1}^{t+1}, \tilde{I}_{t-1}^{t+1}(\mathcal{G}_{t+1})). \quad (8)$$

In addition, we also formulate complementary constraints leveraging the real images. Specifically, we render Gaussians $\mathcal{G}_{t+1}$ from the $(t + 1)$ -th view into an image $\tilde{I}_{t+1}(\mathcal{G}_{t+1})$, which correspond to the real image I_{t+1} , and we define the photometric loss as $\min_{\mathcal{G}_{t+1}} \mathcal{L}_{\text{photo}}(I_{t+1}, \tilde{I}_{t+1}(\mathcal{G}_{t+1}))$.

V. EXPLORATION PLANNING

In this section, we present how we plan a farsighted exploration path. Our main technical contribution is to dream semantically plausible structures of unexplored regions, and then leverage these structures to make decisions. Such a strategy can significantly reduce the path length and improve the exploration completeness. To facilitate understanding, we first briefly introduce the pipeline, and then highlight our contributions.

A. Exploration Planning Pipeline

We formulate the exploration as the online generation of a set of candidate waypoints, and the selection of the optimal one to move to.

Generation of Waypoints. We make use of our 3D scene mapping method, introduced above, to reconstruct the scene with a 3D Gaussian map. To extract traversable areas in the 3D space, we render Gaussians into a top-view opacity image, which is then binarized as a free-space map. We then use Voronoi diagrams [57] to convert this free-space map into a 2D topological map with multiple nodes and edges, the

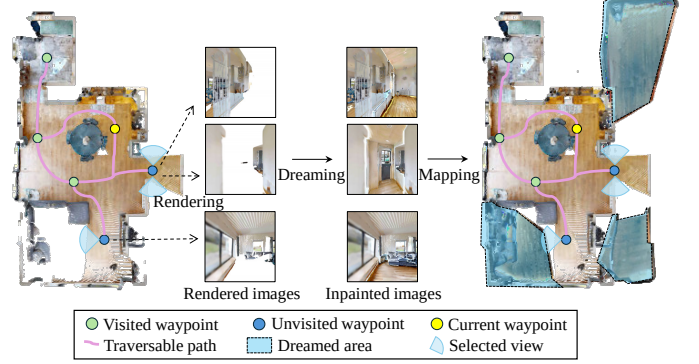


Fig. 5. **Dreaming semantically plausible structures of unexplored areas.** At an unvisited waypoint, we place virtual cameras to render images from different views and select the suitable images. Then we inpaint the selected images, and use them to predict Gaussians. By integrating the dreamed Gaussians into the existing Gaussians, we obtain more complete structures of the environment.

former corresponding to waypoints and the latter representing their connections. As the Gaussian map grows, we update the topological map with newly added nodes.

Motion to Optimal Waypoints. We first cluster waypoints according to their distances in the topological map. Accordingly, the environment is partitioned into several sub-regions, each of which is associated with a representative waypoint. We perform a hierarchical planning by globally ordering all sub-regions and locally exploring each of them. In terms of global ordering, we determine the shortest path to sequentially connect all the representative waypoints of the sub-regions. We cast this planning as a solvable traveling salesman problem [58]. Then the robot moves to the first sub-region in the planned sequence, followed by re-performing the global ordering on both remaining and added sub-regions. The added sub-regions include both newly observed and dreamed sub-regions (sub-region dreaming is our main technical novelty). The reason for re-planning is that the added sub-regions may overturn the optimality of the previous path. In terms of local planning within a sub-region, the robot preferentially moves to the waypoints where the surroundings are under-explored. This strategy targets a quick gain of information on the map. The traversable path between waypoints is computed by [59].

B. Structure Dreaming-based Global Ordering

When performing the global ordering of sub-regions, a common strategy is to leverage the observed structures. However, in this case, many structures are only partially observed, and thus the information is incomplete, resulting in unnecessary detours. To solve this problem, we propose to dream semantically plausible structures of the unobserved areas.

Dreaming Semantically Plausible Structures. As shown in Fig. 5, at each unvisited waypoint within a subregion, we first render the reconstructed Gaussians into opacity $\{\hat{O}_i\}_{i=1}^4$ and RGB images $\{\tilde{I}_i\}_{i=1}^4$ from four non-overlapping views (each of them with a field of view of 90°). These views collectively cover the full panorama around the waypoint. Intuitively, if a pixel of the image $\hat{O}_i$ has a high opacity, it is occupied and corresponds to a mapped structure. For each

image $\tilde{O}_i$ or $\tilde{I}_i$ with N pixels, we compute its occupancy score g_i defined as the proportion of occupied pixels:

$$g_i = \sum_{j=1}^N \mathbb{I}\{\tilde{o}_j > \tau_o\} / N, \quad (9)$$

where $\tilde{o}_j$ denotes the opacity of the j -th pixel of the image $\tilde{O}_i$, τ_o denotes the opacity threshold, and $\mathbb{I}\{\cdot\}$ returns 1 if the condition is satisfied. As to the suitability of the image $\tilde{I}_i$ for structure dreaming, we disregard both too high and too low occupancy scores g_i . Excessively high scores mean that the environment has been well-observed and is unnecessary to dream, while excessively low scores mean that the observations are insufficient to provide reliable semantic references for dreaming. Therefore, we only consider view(s) whose score g_i falls within a range $[\underline{\tau}, \bar{\tau}]$ for inpainting. We inpaint the images $\{\tilde{I}_i\}$ associated with the selected views in a semantically plausible way (details will be introduced in Section V-C).

Global Ordering. Given a set of inpainted images, we first predict new Gaussians using the Gaussian prediction network that we described before. Benefiting from high-quality images, these Gaussians are semantically plausible to describe the underlying structures of the unobserved areas. After that, we integrate the dreamed Gaussians into the existing Gaussian map. Such an enriched map leads to a refined topological map and additional sub-regions. As shown in Fig. 6, especially for long-horizon planning, our approach using the dreamed structures significantly outperforms the shortsighted planning method without the dreaming capability. Our method globally shortens the path length and yields higher mapping efficiency. Please note that while our dreamed structures are semantically plausible and effective for planning, they inevitably differ from the real structures. Accordingly, for the photo-realistic scene mapping, we differentiate between the dreamed and observed sub-regions during exploration. After the robot visits a dreamed sub-region, it uses real observations to update the representation of this sub-region, replacing the previously dreamed structures.

Dynamic Environments. Our planning method can handle dynamic environments well. This is mainly attributed to our reconstructed map, in which we can differentiate between the static background and dynamic foreground. First, existing methods [6], [60] mistakenly reconstruct the foreground as the background, which causes structure blur and further blocks the feasible traversable path. By contrast, our method can distinguish and only use the structures of static background for path planning, avoiding the occlusion of the paths to unexplored sub-regions. Please refer to the supplementary material for details. Second, during the structure dreaming, we only render the static background into images on purpose. This operation contributes to a holistic scene layout prediction without being affected by the foreground occlusion. Third, dynamic objects are typical obstacles for robot movement. Our mapping approach can determine the shapes and relative locations of these dynamic obstacles at each time. By feeding this information to Dijkstra’s algorithm [59], the robot can effectively avoid obstacles.

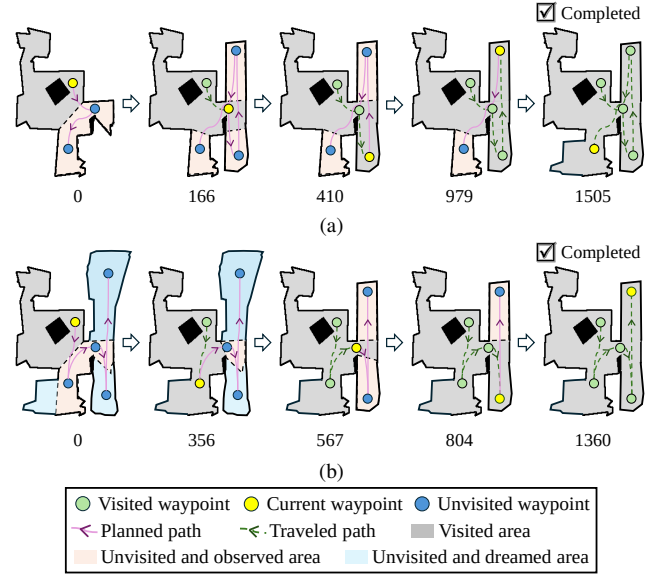


Fig. 6. **Comparison between global ordering strategies without and with the capability of structure dreaming.** In this schematic diagram, the number below each image represents the accumulated length of the traveled path. They do not have a specific unit, but show relative magnitudes of values. (a) Strategies without dreaming capability result in suboptimal trajectories with unnecessary detours. (b) Our method leverages both observed and dreamed structures to plan a farsighted path.

C. Semantically Plausible Image Inpainting

Recall that in the above subsection, we inpaint the rendered images $\{\tilde{I}_i\}$. We achieve this by proposing a network that can effectively leverage the surrounding information to achieve a semantically plausible inpainting. It is based on the diffusion model, which is similar to our inpainting network in Section IV-A. Their main differences lie in the inpainting mask definition and reference code generation.

For one thing, we define an inpainting mask $\mathbf{B}$ to indicate the pixels to inpaint. During training, we randomly generate $\mathbf{B}$ of the complete image $\hat{I}_i$ to improve the generalization of the network. At the inference stage, the inpainted pixels of the rendered image $\tilde{I}_i$ are defined as a set of non-occupied pixels whose opacity is lower than the threshold τ_o (see Eq. (9)). For another, we introduce how we generate the reference code as follows. We use the complementary mask of the above inpainting mask $\mathbf{B}$ to indicate the occupied pixels. During training, given the masked image $\hat{I}_i$, we encode its occupied pixels into the reference code $\mathbf{c}$ based on the pretrained variational autoencoder [52]. At inference time, we use this encoder to encode the occupied pixels of the rendered image $\tilde{I}_i$ as the reference code $\mathbf{c}$. The code $\mathbf{c}$ can provide the structural and appearance information of the surrounding pixels for reliable inpainting.

Given the above inpainting mask $\mathbf{B}$ and reference code $\mathbf{c}$, we introduce the diffusion-based inpainting. In the forward process, we generate the noisy code $\mathbf{z}_s$ at step s . In the reverse process, we introduce a network $\mathcal{P}$ to predict the added noise ϵ . This network incorporates the code $\mathbf{c}$ and the downsampled mask $\bar{\mathbf{B}}$ compatible with the noise dimension:

$$\epsilon = \mathcal{P}(\mathbf{z}_s, s, \mathbf{c}, \bar{\mathbf{B}}). \quad (10)$$

TABLE I
CAMERA LOCALIZATION COMPARISONS ON THE TUM [61] AND BONN [30] DATASETS.

TUM dataset [61]															
Sequences	f3/wk_xyz		f3/wk_hf		f3/wk_st		f3/st_hf		f3/st_rpy		f3/st_st		f3/st_xyz		Average
	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	
ORB-SLAM3 [62]	28.1	12.2	30.5	9.0	2.0	1.1	2.6	1.6	6.4	2.5	0.98	0.46	1.6	0.7	10.31 3.94
MonST3R [35]	26.7	13.2	44.6	23.9	1.7	0.8	38.7	20.2	5.5	2.6	2.2	1.2	30.2	13.2	21.37 10.73
RoDyn-SLAM [17]	8.3	5.5	5.6	2.8	1.7	0.9	4.4	2.2	11.4	4.6	0.76	0.43	5.0	1.0	5.31 2.49
PG-SLAM [18]	6.8	2.9	11.7	4.4	1.4	0.6	4.0	1.5	5.4	2.4	0.72	0.39	1.5	0.5	4.50 1.81
WildGS-SLAM [21]	1.3	0.6	1.6	0.8	0.4	0.2	2.0	0.9	2.4	1.4	0.5	0.3	0.8	0.4	1.28 0.65
Dream-SLAM (ours)	1.7	0.7	1.6	0.7	0.3	0.1	1.9	0.9	2.3	0.7	0.3	0.1	0.6	0.2	1.27 0.48

Bonn dataset [30]															
Sequences	balloon		balloon2		ps_track		ps_track2		mv_box		mv_box2		Average		
	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	RMSE↓	SD↓	
ORB-SLAM3 [62]	6.5	2.9	17.7	8.6	70.7	32.6	77.9	43.8	28.0	8.4	3.5	1.5	34.05	16.30	
MonST3R [35]	11.1	8.5	14.3	6.3	25.4	15.8	22.5	9.4	7.6	3.5	13.1	5.3	15.67	8.13	
RoDyn-SLAM [17]	7.9	2.7	11.5	6.1	14.5	4.6	13.8	3.5	7.2	2.4	12.6	4.7	11.25	4.00	
PG-SLAM [18]	6.4	2.2	7.3	3.4	5.0	1.9	8.5	2.8	4.6	1.3	7.0	2.0	6.47	2.27	
WildGS-SLAM [21]	2.8	1.2	2.4	1.1	3.1	2.0	3.0	1.3	1.6	0.8	2.2	1.2	2.52	1.27	
Dream-SLAM (ours)	1.9	0.7	1.9	0.4	1.5	0.5	2.7	0.8	0.6	0.2	1.4	0.5	1.67	0.52	

To train the network $\mathcal{P}$, we use the following loss:

$$\mathcal{L} = \mathbb{E} \left[\|\bar{\mathbf{B}} \odot (\hat{\epsilon} - \epsilon(\mathcal{P}))\|_2 \right]. \quad (11)$$

During inference, given a noisy code expressed by white noise, we follow the reverse process to obtain the denoised code $\mathbf{z}_0$. By decoding $\mathbf{z}_0$ based on the pretrained decoder [52], we generate the inpainted image. This image exhibits a semantically plausible appearance where the inpainted areas are harmonious with the observed areas.

VI. EXPERIMENTS ON PUBLIC DATASETS

To the best of our knowledge, currently, there is no public dataset that can simultaneously evaluate 1) the accuracy of localization and mapping, together with 2) the effectiveness of exploration planning. Therefore, in this section, we separately evaluate these two tasks. All the experiments are conducted on a server with an E3-1226 CPU and RTX 4090 GPU. For a joint evaluation on our self-collected data, please refer to the next section.

A. Localization and Mapping

1) *Experimental Setup*: We introduce datasets, evaluation metrics, implementation details, and methods for comparison.

Datasets. TUM RGB-D [61] (denoted by TUM) and Bonn RGB-D dynamic [30] (denoted by Bonn) datasets are two well-known datasets to benchmark SLAM methods, especially in dynamic environments:

- The TUM dataset was collected in indoor environments. It contains multiple sequences with one or more people walking around. It provides ground-truth camera trajectories.

- Compared with the TUM dataset, the Bonn dataset additionally includes scenarios involving human-object interactions, such as box carrying. Moreover, human movement on partial sequences is larger.

Evaluation Metrics. To assess camera localization accuracy, we adopt the widely used absolute trajectory error [63], which measures the difference between the estimated and the ground-truth trajectories. We present the error in terms of root mean square (RMSE) [61] and standard deviation (SD), both expressed in units of centimeters. Regarding mapping quality, we evaluate the rendering results. For quantitative evaluation, we render the reconstructed scene from novel viewpoints. For qualitative analysis, we employ widely adopted metrics PSNR [64], SSIM [50], and LPIPS [65] to measure the differences between the rendered and the ground-truth images.

Implementation Details. For the diffusion model introduced in Section IV-A, we adopt a large-scale pretrained text-to-image network [19] and apply LoRA-based fine-tuning with a rank of 8. For our Gaussian prediction network introduced in Section IV-C, we initialize it with the pretrained MonST3R [35] and adopt a two-stage training strategy. In the first stage, we fine-tune the position head and decoder based on the point loss, while in the second stage, we fix the other modules and train the Gaussian head only. As to the data for the above fine-tuning, we consider the Neuman [66] and WildGS [21] datasets, which contain scenes with multiple people and moving objects. This practice is helpful to evaluate the generalization when testing on the TUM and Bonn datasets.

Methods for Comparison. We compare the localization and mapping modules of our Dream-SLAM against the following state-of-the-art SLAM methods introduced in Section II:

- ORB-SLAM3 [62]: A classic feature-based method. It



Fig. 7. **Qualitative comparison of dynamic scene mapping methods on Sequence wk_xyz in the TUM dataset [61].** We render the mapped scenes from novel views. (a) MonST3R [35] does not support a photo-realistic rendering due to the point cloud representation. (b) WildGS-SLAM [21] fails to map the dynamic foreground. (c) PG-SLAM [18] can reconstruct both static background and dynamic foreground, but some rendering regions remain low-quality. (d) Our Dream-SLAM can map both background and foreground, and also achieves the highest mapping quality.

leverages the epipolar geometry constraints to filter out dynamic objects.

- Rodyn-SLAM [17]: An implicit representation-based method designed for dynamic environments. It eliminates the dynamic objects using estimated masks.
- PG-SLAM [18]: A Gaussian splatting-based method suitable for dynamic environments. It reconstructs dynamic objects online, based on a priori motion constraints.
- MonST3R [35]: A geometry-based method that relies on the alignment between point clouds. It can directly output dynamic point clouds.
- WildGS-SLAM [21]: A Gaussian splatting-based method designed for dynamic environments. It eliminates dynamic objects by learning dynamic regions online.

Among them, ORB-SLAM3, Rodyn-SLAM, and PG-SLAM take RGB-D images as input, while MonST3R, WildGS-SLAM, and our Dream-SLAM only use RGB images.

2) *Localization Results:* Table I shows comparisons on both Bonn and TUM datasets. On the Bonn dataset, ORB-SLAM3 shows the highest errors. The reason is that it cannot tolerate large fractions of matches in dynamic objects. Rodyn-SLAM and WildGS-SLAM achieve higher accuracy by exploiting the constraints of background, but disregard foreground informa-

tion. MonST3R considers the alignment constraints of both foreground and background, but heavily depends on the pre-trained model’s outputs, which may not be reliable in practice. PG-SLAM incorporates both foreground and background for camera localization. However, we empirically observed that it cannot effectively leverage foreground objects with large motions. Our Dream-SLAM achieves the highest accuracy by providing virtual observation constraints, enabling more effective use of foreground information.

On the TUM dataset, there are several sequences with relatively small camera and human motions. ORB-SLAM3 achieves satisfactory performance on these sequences. However, on other highly dynamic sequences, it leads to significant errors. Similarly, MonST3R becomes unstable on highly dynamic sequences. By contrast, Rodyn-SLAM maintains good stability by filtering out the foreground. PG-SLAM, which leverages both foreground and background information, achieves better performance. WildGS-SLAM performs relatively well thanks to its uncertainty map-based optimization. Despite this, our Dream-SLAM surpasses it by incorporating cross-spatio-temporal observations and exploiting both foreground and background information for localization.

3) *Mapping Results:* We quantitatively evaluate the rendering performance, as shown in Table II. Since some methods cannot reconstruct the foreground, we compute the evaluation metrics based on two settings: over the entire image and with the foreground regions masked out. We also provide qualitative comparison in novel-view renderings, as illustrated in Fig. 7. To ensure a fair comparison, the reconstructed scenes of different methods are rendered from the same viewpoint.

Rodyn-SLAM merely focuses on background mapping. As a NeRF-based method, it shows a noticeable gap compared to Gaussian splatting-based approaches. WildGS-SLAM cannot reconstruct the foreground, resulting in lower performance when evaluating the full images. Moreover, its learned uncertainty mask is sensitive to noise in practice, resulting in floating artifacts in background rendering. MonST3R adopts 3D points as reconstruction primitives, thus the discrete projections exhibit unsatisfactory quality. PG-SLAM achieves relatively accurate foreground reconstruction. However, for background regions, depth information may be unstable due to the limited measurement range of the depth camera. As a result, PG-SLAM frequently produces distorted reconstructions in far fields or boundary areas. Our Dream-SLAM achieves better performance in both foreground and background reconstructions. The key reason is that it maps the scene by our deep geometry model rather than depth measurements, which avoids the influence of outliers and better preserves the spatial relationships between objects. In addition, our tracking strategy improves reconstruction quality by providing more accurate camera poses.

4) *Efficiency Evaluation:* We compare the per-frame time costs of different methods on a representative sequence mv_box of the Bonn dataset [30] that simultaneously involves non-rigid human, rigid box, and static background. As shown in Table III, our Dream-SLAM achieves the highest efficiency. Among its total runtime of 0.65 s, dreaming cross-spatio-temporal images accounts for about 0.3 s. WildGS-SLAM is

TABLE II
RENDERING QUALITY COMPARISON ON THE BONN [30] AND TUM [61] DATASETS. “W/O” AND “W” REPRESENT RENDERING EVALUATIONS WITHOUT AND WITH CONSIDERING THE DYNAMIC FOREGROUNDS, RESPECTIVELY.

		Bonn dataset [30]						TUM dataset [61]					
		Seq. ps_track2		Seq. ps_track		Seq. mv_box2		Seq. wk_st		Seq. st_hf		Seq. wk_xyz	
		w/o	w	w/o	w	w/o	w	w/o	w	w/o	w	w/o	w
RoDyn-SLAM [17]	PSNR↑	18.46	16.12	18.53	16.13	18.30	17.48	11.48	11.40	11.47	11.11	11.93	11.91
	SSIM↑	0.745	0.656	0.742	0.659	0.731	0.695	0.684	0.522	0.514	0.674	0.567	0.384
	LPIPS↓	0.545	0.649	0.506	0.597	0.571	0.612	0.502	0.663	0.472	0.511	0.570	0.745
WildGS-SLAM [21]	PSNR↑	21.87	18.12	22.13	18.55	21.40	20.13	21.60	16.61	19.49	17.94	17.11	14.44
	SSIM↑	0.827	0.791	0.817	0.787	0.783	0.770	0.844	0.7473	0.745	0.691	0.684	0.617
	LPIPS↓	0.263	0.317	0.296	0.341	0.401	0.421	0.135	0.261	0.255	0.331	0.254	0.370
PG-SLAM [18]	PSNR↑	26.79	26.82	26.68	27.39	27.25	27.38	23.55	25.65	24.96	25.46	21.56	22.99
	SSIM↑	0.955	0.956	0.923	0.958	0.947	0.948	0.939	0.969	0.933	0.941	0.937	0.956
	LPIPS↓	0.166	0.167	0.179	0.157	0.195	0.191	0.155	0.073	0.170	0.162	0.125	0.096
Dream-SLAM (ours)	PSNR↑	29.50	29.69	27.18	27.45	26.60	26.71	25.92	26.26	25.06	25.73	22.42	23.31
	SSIM↑	0.957	0.960	0.911	0.917	0.957	0.958	0.964	0.971	0.942	0.951	0.933	0.945
	LPIPS↓	0.077	0.072	0.158	0.149	0.097	0.095	0.091	0.067	0.106	0.090	0.095	0.079

TABLE III
EFFICIENCY COMPARISON IN TERMS OF LOCALIZATION AND MAPPING ON SEQUENCE mv_box IN THE BONN DATASET [30].

	Time Cost Per Frame
WildGS-SLAM [21]	2.79 s
PG-SLAM [18]	1.93 s
Rodyn-SLAM [17]	1.42 s
Dream-SLAM (ours)	0.65 s

the most time-consuming due to the relatively complex uncertainty map prediction. Both Rodyn-SLAM and PG-SLAM need to train the mapping networks in an online manner, which are slower than our feedforward 3D mapping network. In addition, for localization, our Dream-SLAM combines both geometric and photometric constraints, leading to a smaller number of optimization iterations than the other methods.

B. Exploration Planning

1) *Experimental Setup*: We introduce datasets, evaluation metrics, implementation details, and methods for comparison.

Datasets. We conduct experiments on the widely used Gibson [67] and HM3D [68] datasets:

- Gibson dataset was established based on the real-world indoor data. These data are processed by the Habitat simulator [69] to generate observations from arbitrary positions within the scene.
- HM3D dataset has a similar establishment pipeline to that of the Gibson dataset. The main difference is that the HM3D dataset features larger and more complicated scenes than the Gibson dataset.

The quality of images provided by the above datasets is relatively low, which makes them unsuitable for localization and

mapping evaluation. Please note that these datasets originally do not contain dynamic humans. To evaluate the performance of the algorithms in dynamic environments, we follow [70] to add dynamic humans into rooms on purpose. We conduct experiments in both dynamic and static environments.

Evaluation Metrics. For quantitative evaluation, we follow [6] to compute the total path length (PL) in units of meters, and reconstruction completeness ratio (%) defined by the coverage of the reconstructed area against the complete area. We denote this completeness ratio obtained in the dynamic and static environments by CR* and CR, respectively. Please note that we empirically observed that state-of-the-art active SLAM methods fail to explore the entire dynamic environments (reasons are introduced in the following text and supplementary material). Accordingly, PL in dynamic environments is not applicable to these methods, and thus we only report PL in static scenes. For qualitative analysis, we provide top-view rendering results of the maps, illustrating the explored areas and traversed paths at different exploration progress.

Implementation Details. For the diffusion model to dream structures of unexplored areas (introduced in Section V-C), we initialize it with the pretrained Stable Diffusion v2 [19] and fine-tune it using LoRA with a rank of 8. To establish the training set for the above fine-tuning, we use the scenes from the Matterport3D dataset [71], which do not overlap with the scenes of the HM3D dataset. This practice is helpful to evaluate the generalization when testing on the HM3D and Gibson datasets. We set the pixel opacity threshold τ_o to 0.5, and set the occupancy score thresholds $\underline{\tau}$ and $\bar{\tau}$ to 0.2 and 0.5, respectively (see Section V-B). We empirically observed that our method is robust to these hyperparameters.

Methods for Comparison. We compare the planning module of our Dream-SLAM against the following state-of-the-art active SLAM methods introduced in Section II:

TABLE IV
EXPLORATION PLANNING COMPARISON ON GIBSON [67] AND HM3D [68] DATASETS. “-” REPRESENTS THE FAILURE OF THE FULL EXPLORATION.

Gibson dataset [67]																							
Sequences	CanWell			Eastville			Swormville			Aloha			Nicut			Quanticco			Average				
	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓		
ANM [7]	-	-	-	47.7	85.0	102.8	44.5	60.5	70.9	32.8	82.5	90.8	46.7	77.4	85.7	53.0	82.1	90.6	44.9	77.5	88.2		
ANM-S [60]	70.3	97.0	91.8	64.4	91.5	70.6	74.8	93.6	95.2	63.6	95.9	75.8	72.4	97.3	83.0	63.5	95.4	79.2	68.1	95.1	82.6		
ActiveSplat [6]	64.8	97.1	116.2	71.9	94.7	84.8	74.7	95.3	91.9	71.3	91.8	82.5	88.9	97.8	94.1	43.6	95.1	93.7	69.2	95.3	93.9		
Dream-SLAM (ours)	98.0	98.1	87.6	95.4	95.3	57.1	95.9	95.9	51.6	98.2	98.1	63.1	98.0	98.1	64.4	95.6	95.6	62.0	96.8	96.9	64.3		

HM3D dataset [68]																							
Sequences	CETmJJqkhcK			7dmR22gwQpH			7UdY7HiDnUi			4h4JxvG3cip			6HMiyl5cxis			T7nCRmufFNR			Average				
	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓	CR*↑	CR↑	PL↓		
ANM [7]	28.3	64.9	112.7	-	-	-	40.3	42.4	92.6	36.8	56.0	142.3	39.5	61.6	144.6	49.5	68.9	107.9	38.9	58.7	120.0		
ANM-S [60]	33.0	92.1	97.4	36.7	90.2	159.2	24.1	91.9	132.1	49.3	93.1	99.5	58.2	95.1	108.6	86.9	95.4	132.2	48.0	92.9	121.5		
ActiveSplat [6]	30.6	92.9	102.9	81.3	94.1	180.0	63.7	95.2	99.8	54.3	92.2	118.8	76.5	95.4	114.6	82.9	94.8	105.37	64.9	94.2	125.8		
Dream-SLAM (ours)	93.2	93.2	84.7	95.0	95.2	154.7	95.9	96.0	75.8	95.3	95.1	89.6	95.7	95.7	85.9	96.9	96.8	75.8	95.3	95.3	99.4		

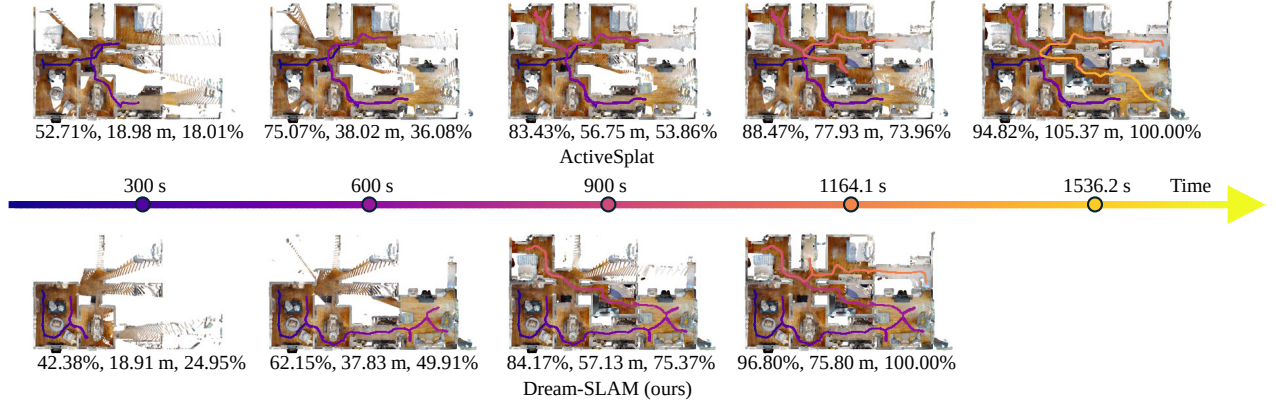


Fig. 8. Comparison between ActiveSplat [6] and our Dream-SLAM in terms of exploration progress on Sequence T7nCRmuffFNR of the HM3D dataset [68]. We provide a top-view visualization of the mapped scenes at some representative timestamps. A triplet of numbers below each image indicates CR, the length of the traversed path, and the proportion of the traversed path to the total path. The trajectory color reflects the time cost of exploration.

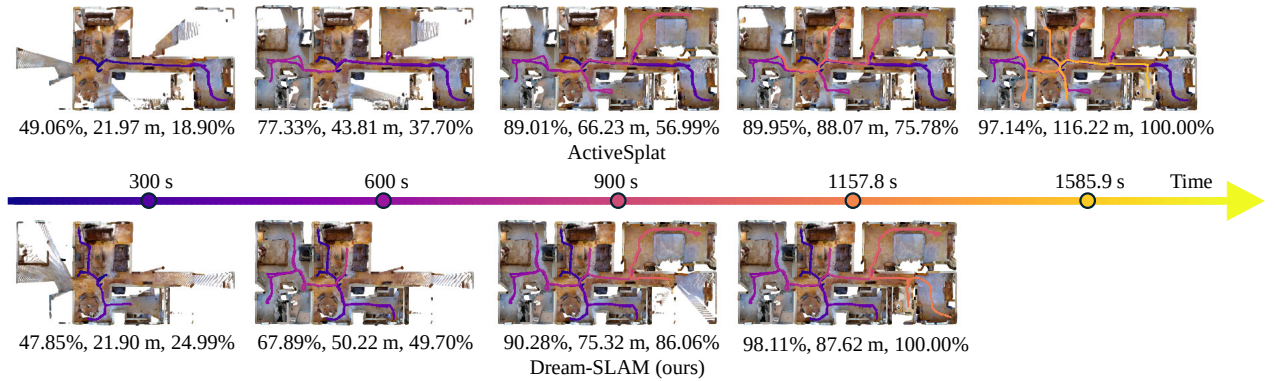


Fig. 9. Comparison between ActiveSplat [6] and our Dream-SLAM in terms of exploration progress on Sequence Cantwell of the Gibson dataset [67]. We provide a top-view visualization of the mapped scenes at some representative timestamps. A triplet of numbers below each image indicates CR, the length of the traversed path, and the proportion of the traversed path to the total path. The trajectory color reflects the time cost of exploration.

TABLE V
ABLATION STUDY OF LOCALIZATION USING THE DREAMED
CROSS-SPATIO-TEMPORAL IMAGES.

Sequence	Without Dreaming		Dreaming (ours)	
	RMSE↓	SD↓	RMSE↓	SD↓
wk_hf of TUM [61]	1.8	0.7	1.6	0.7
ps_track of Bonn [30]	1.7	0.7	1.5	0.5

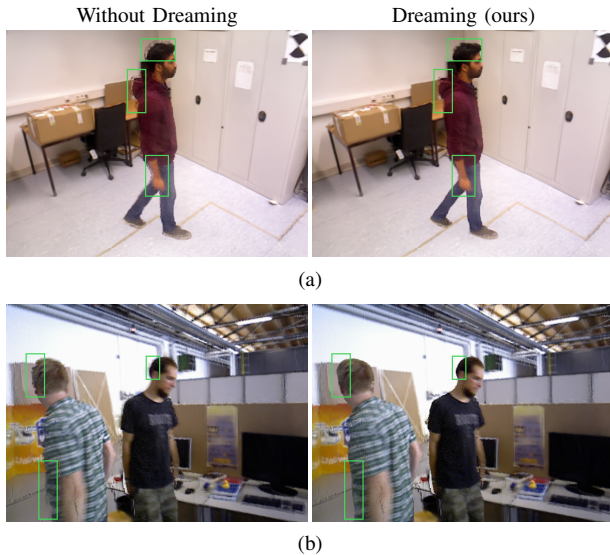


Fig. 10. **Ablation study of mapping using the dreamed cross-spatio-temporal images.** We compare the rendering results of Gaussian maps obtained without and with the dreamed images on (a) Sequence `ps_track` of the Bonn dataset [30], and (b) Sequence `wk_hf` of the TUM dataset [61].

- ANM [7]: A neural representation-based method that determines the target goal by minimizing map uncertainty. It is a classic greedy strategy.
- ANM-S [60]: An implicit representation-based method that is an improvement of the above ANM. It performs planning based on a generalized Voronoi graph.
- ActiveSplat [6]: A Gaussian splatting-based method that is an improvement of the above ANM-S. It prioritizes the exploration of local regions.

2) *Evaluation Results*: The quantitative evaluations in both dynamic and static environments are presented in Table IV. We also provide qualitative visualization of the exploration process in static environments, as shown in Figs. 8 and 9. Overall, existing methods fail to explore the entire dynamic scenes, leading to CR^* significantly smaller than CR . The main reason is that they typically treat dynamic humans as obstacles that occlude the doorways and entrances to other rooms. By contrast, our Dream-SLAM can map dynamic humans and static background separately, and thus is not affected by the dynamic humans during planning. We provide a more detailed analysis in terms of CR and PL as follows. On both datasets, ANM exhibits unsatisfactory performance since its waypoint selection strategy based on map uncertainty tends to overlook certain areas. ANM-S achieves relatively high exploration completeness but with longer paths. Its Voronoi graph-based planning strategy prioritizes the waypoints with

TABLE VI
ABLATION STUDY OF PLANNING USING THE DREAMED STRUCTURES OF
UNEXPLORED AREAS.

Sequence	Without Dreaming		Dreaming (ours)	
	CR↑	PL↓	CR↑	PL↓
Cantwell of Gibson [67]	97.5	101.3	98.1	88.4
T7nCRmufFNR of HM3D [68]	95.5	85.1	96.8	75.8

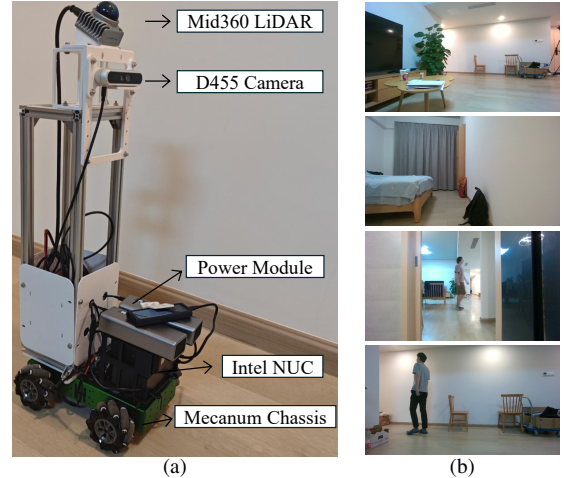


Fig. 11. **Setup of experiments on our self-collected data.** (a) Mobile robot to collect data. It is built on a four-wheeled Mecanum chassis and equipped with a front-facing camera and a LiDAR. (b) Sample images of our self-collected data in a home environment containing dynamic humans.

high uncertainty, resulting in a lack of global planning and frequent backtracking. While ActiveSplat achieves higher exploration completeness by visiting all the waypoints within each sub-region, its path cost further increases as a result. Its decisions tend to be locally optimal when dealing with multi-connected waypoints. By contrast, our Dream-SLAM provides the highest exploration completeness and the lowest path cost across all sequences of both datasets, benefiting from our farsighted planning that reasons over the semantically plausible structures dreamed for unexplored areas.

C. Ablation Study

We conduct ablation study to validate the effectiveness of the proposed modules and strategies.

1) *Localization Using Dreamed Images*: Recall that for camera pose estimation, we leverage the foreground constraints provided by the dreamed cross-spatio-temporal images. We compare our strategy to the version without using such images. As shown in Table V, our strategy achieves higher accuracy. This validates that our image dreaming is reliable, and also the foreground information can effectively enhance the localization accuracy.

2) *Mapping Using Dreamed Images*: Recall that for Gaussian map refinement, we incorporate the dreamed cross-spatio-temporal images as a supplement to real observations. We compare our strategy to the version without using such images. As shown in Fig. 10, we report the qualitative results

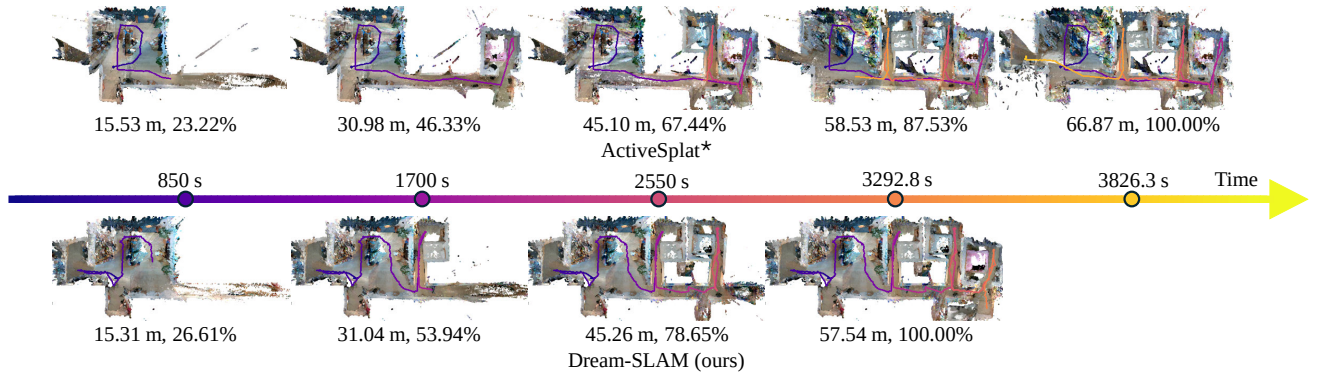


Fig. 12. **Comparison between ActiveSplat* [6], [21] and our Dream-SLAM in terms of exploration progress on our self-collected data.** We provide a top-view visualization of the mapped scenes at some representative timestamps. A pair of numbers below each image indicates the length of the traversed path, and the proportion of the traversed path to the total path. The trajectory color reflects the time cost of exploration.

of dynamic map rendering.³ Results demonstrate that cross-spatio-temporal images can provide additional constraints on foreground reconstruction, and thus reconstruct a more spatially reasonable map.

3) *Planning Using Dreamed Structures:* Recall that we plan the path by dreaming semantically plausible structures of unexplored areas as a supplement to the observed structures. For validation, we compare our method to the version without the dreamed structures. As shown in Table VI, reasoning over semantically plausible structures enables thorough exploration with reduced path length, thereby yielding superior exploration efficiency.

VII. EXPERIMENTS ON SELF-COLLECTED DATA

To jointly evaluate localization and mapping, together with exploration planning, we use the self-collected data obtained in the real world.

1) *Experimental Setup:* As shown in Fig. 11(a), we develop our mobile robot based on a four-wheeled Mecanum chassis. The robot integrates a front-facing D455 camera to obtain RGB images and a Livox Mid-360 LiDAR to acquire ground-truth robot trajectories. Our robot is not equipped with dedicated graphical computing resources. Instead, computation is performed on a server with an NVIDIA RTX 4090 GPU connected via a local network. For robot movement, we set the maximum linear speed to 0.2 m/s and the maximum angular speed to 0.2 rad/s. As shown in Fig. 11(b), we collect data in a home, which includes a living room, a kitchen, three bedrooms, and two bathrooms. The home contains several humans moving around.

We choose the state-of-the-art active SLAM method ActiveSplat as the comparative approach. As mentioned above, the original ActiveSplat uses the ground-truth camera poses provided by the simulator, and performs mapping by SplatTAM [72] that is only suitable for static environments. For a fair comparison, we integrate ActiveSplat with the aforementioned state-of-the-art localization method WildGS-SLAM applicable to dynamic scenes. We denote this integra-



Fig. 13. **Dynamic scene mapping of our Dream-SLAM on our self-collected data.** We render the reconstructed map composed of both static background and dynamic foreground from a novel view. The red and green dotted lines denote the trajectories of human and camera, respectively.

TABLE VII
CAMERA LOCALIZATION COMPARISON BETWEEN WILDGS-SLAM [21] AND OUR DREAM-SLAM ON OUR SELF-COLLECTED DATA.

WildGS-SLAM [21]		Dream-SLAM (ours)	
RMSE↓	SD↓	RMSE↓	SD↓
16.6	7.3	10.8	5.6

tion by ActiveSplat*. Both ActiveSplat* and our Dream-SLAM perform exploration planning from the same starting point. In addition, we independently evaluate the localization by comparing WildGS-SLAM and our Dream-SLAM. For an unbiased comparison, both methods use the same pre-recorded image sequence. This sequence is obtained along the path planned by our Dream-SLAM. We adopt the metrics introduced in Section VI for quantitative evaluation.

2) *Experimental Results:* Fig. 12 presents the comparison in terms of exploration planning. To complete the full exploration, our Dream-SLAM travels a total distance of 57.54 m, saving 14% than ActiveSplat*. Specifically, ActiveSplat* can only exploit the currently observed information and is prone to generating a locally optimal path. For example, it overlooks the kitchen at the beginning and fails to detect several bedrooms later. By contrast, by reasoning over semantically plausible structures imagined for unexplored regions,

³Due to the lack of ground-truth structures of dynamic environments, a quantitative evaluation is infeasible.

our Dream-SLAM plans a more farsighted path. This superior performance validates the effectiveness of our strategy and its adaptability to real-world environments. Moreover, Fig. 13 shows that during planning, our Dream-SLAM can reconstruct the dynamic foreground at different times. This avoids the occlusion of the entrances and doorways to the other rooms. In addition, the independent comparison in terms of camera localization is shown in Table VII. Our Dream-SLAM significantly outperforms WildGS-SLAM. With more accurate camera poses, our Dream-SLAM is better equipped to perform downstream path planning.

VIII. CONCLUSIONS

In this paper, we propose Dream-SLAM, a monocular active SLAM method via dreaming cross-spatio-temporal images and semantically plausible structures of unexplored areas in dynamic environments. For localization, our method uses the dreamed images to formulate novel constraints, simultaneously exploiting dynamic foreground and static background cues. For mapping, we introduce a feedforward network to predict Gaussians and also use the dreamed images for Gaussian refinement, achieving a photo-realistic and coherent representation of both foreground and background. For exploration planning, we propose a strategy that integrates the dreamed structures with the observed geometry, enabling long-horizon reasoning and producing farsighted trajectories. Extensive experiments on public and self-collected datasets demonstrate that our method outperforms state-of-the-art approaches.

REFERENCES

- [1] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.
- [2] J. A. Placed, J. Strader, H. Carrillo, N. Atanasov, V. Indelman, L. Carlone, and J. A. Castellanos, "A survey on active simultaneous localization and mapping: State of the art and new frontiers," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1686–1705, Jun. 2023.
- [3] D. Brugali, L. Muratore, and A. De Luca, "Mobile robots exploration strategies and requirements: A systematic mapping study," *The International Journal of Robotics Research*, vol. 44, no. 9, pp. 1461–1506, Aug. 2025.
- [4] C. Cao, H. Zhu, H. Choset, and J. Zhang, "TARE: A Hierarchical Framework for Efficiently Exploring Complex 3D Environments," in *Robotics: Science and Systems*, 2021, p. 2.
- [5] J. Yu, H. Shen, J. Xu, and T. Zhang, "ECHO: An efficient heuristic viewpoint determination method on frontier-based autonomous exploration for quadrotors," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 5047–5054, Aug. 2023.
- [6] Y. Li, Z. Kuang, T. Li, Q. Hao, Z. Yan, G. Zhou, and S. Zhang, "ActiveSplat: High-Fidelity Scene Reconstruction Through Active Gaussian Splatting," *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 8099–8106, Aug. 2025.
- [7] Z. Yan, H. Yang, and H. Zha, "Active neural mapping," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 10981–10992.
- [8] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.
- [9] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding Horizon "Next-Best-View" Planner for 3D Exploration," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2016, pp. 1462–1468.
- [10] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [11] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, Mar. 2017.
- [12] E. M. Lee, J. Choi, H. Lim, and H. Myung, "REAL: Rapid Exploration with Active Loop-Closing toward Large-Scale 3D Mapping using UAVs," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021, pp. 4194–4198.
- [13] K. Naveed, W. Hussain, I. Hussain, D. Lee, and M. L. Anjum, "Help Me Through: Imitation Learning Based Active View Planning to Avoid SLAM Tracking Failures," *IEEE Transactions on Robotics*, vol. 41, pp. 4236–4252, Jun. 2025.
- [14] C. Ho, S. Kim, B. Moon, A. Parandekar, N. Harutyunyan, C. Wang, K. Sycara, G. Best, and S. Scherer, "MapEx: Indoor Structure Exploration with Probabilistic Information Gain from Global Map Predictions," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 13074–13080.
- [15] B. Zhou, H. Xu, and S. Shen, "RACER: Rapid Collaborative Exploration With a Decentralized Multi-UAV System," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1816–1835, Jun. 2023.
- [16] J. Zhang, M. Henein, R. Mahony, and V. Ila, "VDO-SLAM: A visual dynamic object-aware SLAM system," *arXiv preprint arXiv:2005.11052*, 2020.
- [17] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, "RoDyn-SLAM: Robust Dynamic Dense RGB-D SLAM with Neural Radiance Fields," *IEEE Robotics and Automation Letters*, vol. 9, no. 9, pp. 7509–7516, Sept. 2024.
- [18] H. Li, X. Meng, X. Zuo, Z. Liu, H. Wang, and D. Cremers, "PG-SLAM: Photorealistic and Geometry-Aware RGB-D SLAM in Dynamic Environments," *IEEE Transactions on Robotics*, vol. 41, pp. 6084–6101, Oct. 2025.
- [19] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10684–10695.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, Jul. 2023.
- [21] J. Zheng, Z. Zhu, V. Bieri, M. Pollefeys, S. Peng, and I. Armeni, "WildGS-SLAM: Monocular Gaussian Splatting SLAM in Dynamic Environments," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 11461–11471.
- [22] Y. Xu, H. Jiang, Z. Xiao, J. Feng, and L. Zhang, "DG-SLAM: Robust Dynamic Gaussian Splatting SLAM with Hybrid Pose Optimization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 51577–51596, 2024.
- [23] W. Jiang, B. Lei, and K. Daniilidis, "FisherRF: Active View Selection and Mapping with Radiance Fields Using Fisher Information," in *Proceedings of the European Conference on Computer Vision*, 2024, pp. 422–440.
- [24] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [25] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-Time Single Camera SLAM," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1052–1067, Jul. 2007.
- [26] T. Whelan, R. F. Salas-Moreno, B. Glocker, A. J. Davison, and S. Leutenegger, "ElasticFusion: Real-time dense SLAM and light source estimation," *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, Sept. 2016.
- [27] C. Kerl, J. Sturm, and D. Cremers, "Dense visual SLAM for RGB-D cameras," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013, pp. 2100–2106.
- [28] Z. Teed and J. Deng, "DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras," *Advances in neural information processing systems*, vol. 34, pp. 16558–16569, 2021.
- [29] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, "DynSLAM: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [30] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, "ReFusion: 3D Reconstruction in Dynamic Environments for RGB-D Cameras Exploiting Residuals," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2019, pp. 7855–7862.
- [31] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, "Flowfusion: Dynamic dense rgb-d slam based on optical flow," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2020, pp. 7322–7328.

- [32] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018, pp. 1168–1174.
- [33] Y. Qiu, C. Wang, W. Wang, M. Henein, and S. Scherer, “AirDOS: Dynamic SLAM benefits from Articulated Objects,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2022, pp. 8047–8053.
- [34] D. F. Henning, T. Laidlow, and S. Leutenegger, “BodySLAM: Joint Camera Localisation, Mapping, and Human Motion Tracking,” in *Proceedings of the European Conference on Computer Vision*, 2022, pp. 656–673.
- [35] J. Zhang, C. Herrmann, J. Hur, V. Jampani, T. Darrell, F. Cole, D. Sun, and M.-H. Yang, “MonST3R: A Simple Approach for Estimating Geometry in the Presence of Motion,” in *Proceedings of the International Conference on Learning Representations*, 2025.
- [36] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *Proceedings of the European Conference on Computer Vision*, 2020, pp. 405–421.
- [37] T. Deng, Y. Pan, S. Yuan, D. Li, C. Wang, M. Li, L. Chen, L. Xie, D. Wang, J. Wang *et al.*, “What is the best 3d scene representation for robotics? from geometric to foundation models,” *arXiv preprint arXiv:2512.03422*, 2025.
- [38] M. M. Johari, C. Carta, and F. Fleuret, “ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 408–17 419.
- [39] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, “iMAP: Implicit Mapping and Positioning in Real-Time,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6229–6238.
- [40] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian Splatting SLAM,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 039–18 048.
- [41] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, “Gaussian-SLAM: Photo-realistic Dense SLAM with Gaussian Splatting,” *arXiv preprint arXiv:2312.10070*, 2023.
- [42] L. Jin, X. Zhong, Y. Pan, J. Behley, C. Stachniss, and M. Popović, “ActiveGS: Active Scene Reconstruction Using Gaussian Splatting,” *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4866–4873, May 2025.
- [43] L. Chen, H. Zhan, K. Chen, X. Xu, Q. Yan, C. Cai, and Y. Xu, “ActiveGAMER: Active GAussian Mapping through Efficient Rendering,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 16 486–16 497.
- [44] M. Juliá, A. Gil, and O. Reinoso, “A comparison of path planning strategies for autonomous exploration and mapping of unknown environments,” *Autonomous Robots*, vol. 33, no. 4, pp. 427–444, 2012.
- [45] T. Cieslewski, E. Kaufmann, and D. Scaramuzza, “Rapid exploration with multi-rotors: A frontier selection method for high speed flight,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017, pp. 2135–2142.
- [46] S. Shen, N. Michael, and V. Kumar, “Stochastic differential equation-based exploration algorithm for autonomous indoor 3d exploration with a micro-aerial vehicle,” *The International Journal of Robotics Research*, vol. 31, no. 12, pp. 1431–1444, Dec. 2012.
- [47] B. Zhou, Y. Zhang, X. Chen, and S. Shen, “FUEL: Fast UAV Exploration Using Incremental Frontier Structure and Hierarchical Planning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 779–786, Apr. 2021.
- [48] Y. Zhang, X. Chen, C. Feng, B. Zhou, and S. Shen, “FALCON: Fast Autonomous Aerial Exploration Using Coverage Path Guidance,” *IEEE Transactions on Robotics*, vol. 41, pp. 1365–1385, 2025.
- [49] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “DUST3R: Geometric 3D Vision Made Easy,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 20 697–20 709.
- [50] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [51] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [52] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *Proceedings of the International Conference on Learning Representations*, 2014.
- [53] Z. Chen, J. Yang, J. Huang, R. de Lutio, J. M. Esturo, B. Ivanovic, O. Litany, Z. Gojcic, S. Fidler, M. Pavone, L. Song, and Y. Wang, “OmniRe: Omni urban scene reconstruction,” in *Proceedings of the International Conference on Learning Representations*, 2025.
- [54] K. S. Arun, T. S. Huang, and S. D. Blostein, “Least-Squares Fitting of Two 3-D Point Sets,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 5, pp. 698–700, Sept. 1987.
- [55] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 16 695–16 705.
- [56] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, “Point transformer v3: Simpler, faster, stronger,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 4840–4851.
- [57] A. Okabe, B. Boots, K. Sugihara, and S. Chiu, *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, ser. Wiley Series in Probability and Statistics. Wiley, 2009.
- [58] N. M. Razali, J. Geraghty *et al.*, “Genetic algorithm performance with different selection strategies in solving tsp,” in *Proceedings of the world congress on engineering*, 2011, pp. 1–6.
- [59] E. W. Dijkstra, “A note on two problems in connexion with graphs,” in *Edsger Wybe Dijkstra: his life, work, and legacy*, 2022, pp. 287–290.
- [60] Z. Kuang, Z. Yan, H. Zhao, G. Zhou, and H. Zha, “Active neural mapping at scale,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 7152–7159.
- [61] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of RGB-D SLAM systems,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 573–580.
- [62] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [63] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual-(Inertial) Odometry,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018, pp. 7244–7251.
- [64] A. Hore and D. Ziou, “Image quality metrics: Psnr vs. ssim,” in *International Conference on Pattern Recognition*, 2010, pp. 2366–2369.
- [65] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 586–595.
- [66] W. Jiang, K. M. Yi, G. Samei, O. Tuzel, and A. Ranjan, “NeuMan: Neural Human Radiance Field from a Single Video,” in *Proceedings of the European Conference on Computer Vision*, 2022, pp. 402–418.
- [67] F. Xia, A. R. Zamir, Z. He, A. Sax, J. Malik, and S. Savarese, “Gibson Env: Real-World Perception for Embodied Agents,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9068–9079.
- [68] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. M. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, “Habitat-Matterport 3D Dataset (HM3D): 1000 Large-scale 3D Environments for Embodied AI,” in *Proceedings of the conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [69] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik *et al.*, “Habitat: A platform for embodied ai research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
- [70] X. Puig, E. Undersander, A. Szot, M. D. Cote, T.-Y. Yang, R. Partsey, R. Desai, A. Clegg, M. Hlavac, S. Y. Min, V. Vondruš, T. Gervet, V.-P. Berges, J. M. Turner, O. Maksymets, Z. Kira, M. Kalakrishnan, J. Malik, D. S. Chappot, U. Jain, D. Batra, A. Rai, and R. Mottaghi, “Habitat 3.0: A Co-Habitat for Humans, Avatars, and Robots,” in *Proceedings of the International Conference on Learning Representations*, 2024.
- [71] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Niebner, M. Savva, S. Song, A. Zeng, and Y. Zhang, “Matterport3D: Learning from RGB-D Data in Indoor Environments,” in *Proceedings of the International Conference on 3D Vision*, 2017, pp. 667–676.
- [72] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “SplatTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.